

## Experiment-04:

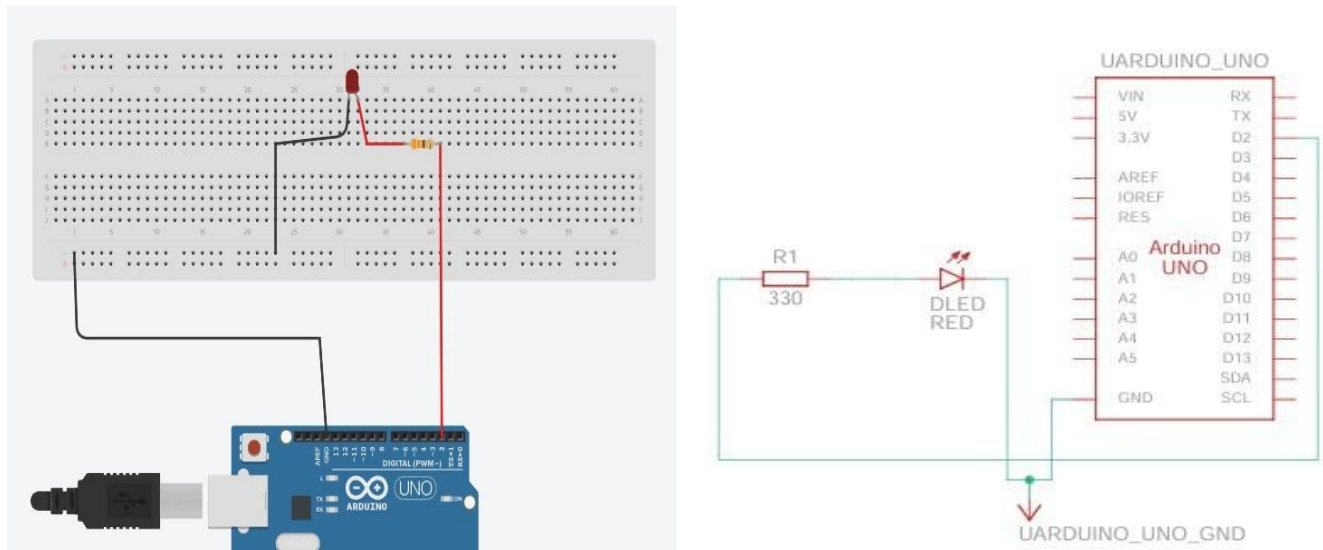
**Name of the Experiment:** Arduino project-I (Blinking LED light)

### Components Required:

- 1 × Breadboard
- 1 × Arduino Uno R3
- 1 × LED
- 1 × 330Ω Resistor
- 2 × Jumper
- USB loader For loading code.
- Arduino IDE.

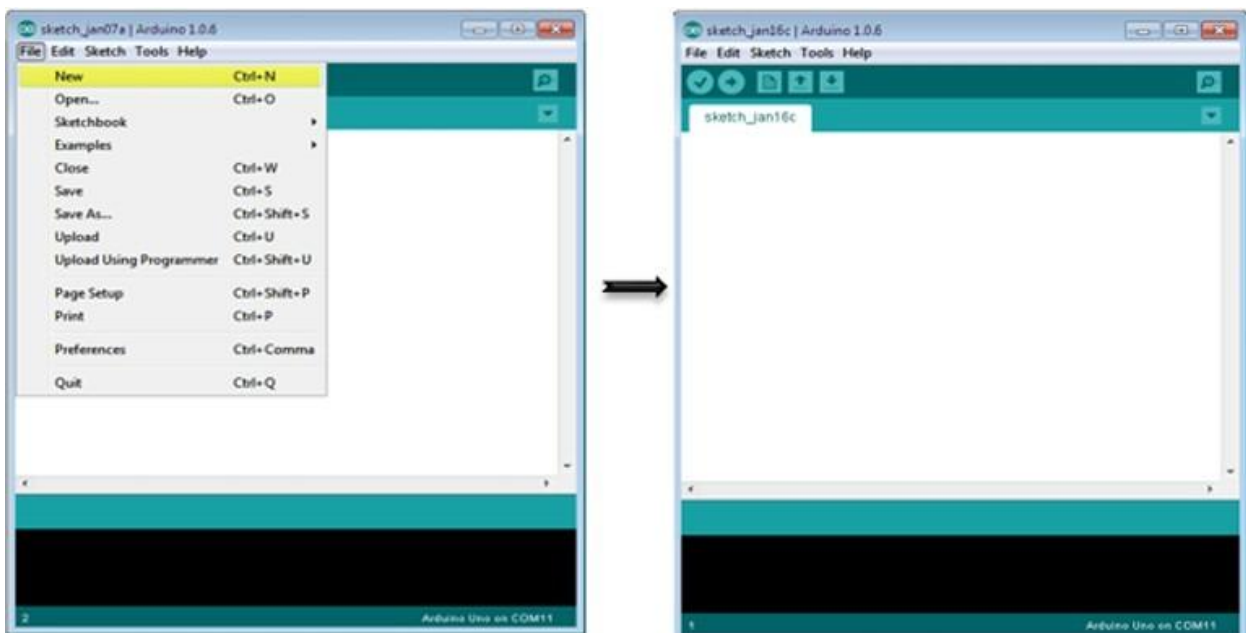
### Procedure

Follow the circuit diagram and hook up the components on the breadboard as shown in the image given below.



**Fig 01:** Arduino Blinking LED Project

Open the Arduino IDE software on your computer. Coding in the Arduino language will control your circuit. Open the new sketch File by clicking New.



**Fig 02:** Browsing Arduino

**Code:**

```
// C++ code
//
void setup()
{
  pinMode(2, OUTPUT);
}

void loop()
{
  digitalWrite(2, HIGH);
  delay(1000); // Wait for 1000 millisecond(s)
  digitalWrite(2, LOW);
  delay(1000); // Wait for 1000 millisecond(s)
}
```

## **Result:**

In this experiment, an Arduino Uno was successfully programmed to control an LED in a blinking pattern. After uploading the program to the Arduino board, the LED connected to the digital output pin turned **ON and OFF repeatedly** with a fixed time delay.

The LED glowed for the specified delay period and then turned off for the same duration, creating a continuous blinking effect. The observed behavior of the LED exactly matched the logic written in the Arduino sketch using the `digitalWrite()` and `delay()` functions.

The Arduino Uno correctly supplied the required voltage to the LED through the output pin, and the current-limiting resistor protected the LED from damage. No abnormal behavior or malfunction was observed during the operation. Hence, the objective of blinking an LED using Arduino Uno was achieved successfully.

## **Discussion:**

The blinking LED project demonstrates the basic working principle of the Arduino Uno and its ability to control external electronic components using digital output pins. The `pinMode()` function was used to configure the pin as an output, while the `digitalWrite()` function controlled the HIGH and LOW states of the LED.

The delay between ON and OFF states was controlled using the `delay()` function, which pauses the program execution for a specified number of milliseconds. Changing the delay value directly affected the blinking speed, proving that timing control in Arduino is simple and flexible.

This experiment helped in understanding fundamental concepts such as digital output, program execution flow, and hardware–software interaction. It also showed the importance of using a current-limiting resistor to ensure safe operation of the LED.

Overall, the experiment provides a strong foundation for more advanced Arduino projects such as traffic light systems, sensor-based automation, and IoT applications.